

Experiment - 08

Student Name: Kamal Sharma
Branch: CSE - AIML
Semester: 5
Subject Name: ADBMS

UID: 24BAI70380
Section/Group: 24AIT_KRG-1_B
Date of Performance: 07-09-26
Subject Code: 24CSP-310

A)

A company maintains customer information, product information, and sales order information in three separate tables: `customer_master`, `product_catalog`, and `sales_orders`.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount.

For every order, first calculate the Gross Amount using:

$\text{Gross Amount} = \text{Unit Price} \times \text{Quantity}$

Then calculate the Discount Amount using:

$\text{Discount Amount} = \text{Gross Amount} \times \text{Discount Percentage} / 100$

Finally, calculate the Final Amount using:

$\text{Final Amount} = \text{Gross Amount} - \text{Discount Amount}$

Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named `client_user` and give this user only `SELECT` permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

Output:

order_id	order_date	full_name	product_name	final_amount
1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Solution

CREATE OR REPLACE VIEW vw_FINAL_AMOUNT_CALCULATION AS
SELECT

S.ORDER_ID AS ORDER_ID,

S.ORDER_DATE AS ORDER_DATE,

C.FULL_NAME AS FULL_NAME,

P.PRODUCT_NAME AS PRODUCT_NAME,

ROUND(P.UNIT_PRICE*S.QUANTITY-

P.UNIT_PRICE*S.QUANTITY*S.DISCOUNT_PERCENT/100,2) AS

FINAL_AMOUNT

FROM SALES_ORDERS AS S

JOIN CUSTOMER_MASTER AS C

ON S.CUSTOMER_ID=C.CUSTOMER_ID

JOIN PRODUCT_CATALOG AS P

ON S.PRODUCT_ID=P.PRODUCT_ID;

SELECT * FROM VW_FINAL_AMOUNT_CALCULATION;

```
CREATE USER CLIENT_123
```

```
PASSWORD '12345';
```

```
GRANT SELECT ON vW_FINAL_AMOUNT_CALCULATION TO  
CLIENT_123;
```

B)

An e-commerce company wants to regularly analyze the performance of its product categories.

The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL Materialized View named: mv_category_sales

Also demonstrate that when new order data is inserted, the Materialized View does not automatically reflect the change and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

OUTPUT

category_id	category_name	total_products_sold	total_orders	unique_customers	total_revenue	avg_revenue_per_order	category_performance
C1	Electronics	5	4	3	12000	3000.00	Excellent
C2	Clothing	4	3	2	10000	3333.33	Excellent
C3	Books	3	1	1	1500	1500.00	Low

Solution

```
CREATE MATERIALIZED VIEW MV_CATEGORY_SALE AS
```

```
SELECT
```

```
    SUM(O.QUANTITY) AS TOTAL_PRODUCTS_SOLD,
```

```
    COUNT(DISTINCT O.ORDER_ID) AS TOTAL_ORDERS,
```

```
    COUNT(DISTINCT O.CUSTOMER_ID) AS UNIQUE_CUSTOMERS,
```

```
SUM(O.QUANTITY*O.UNIT_PRICE) AS TOTAL_REVENUE,  
ROUND(SUM(O.QUANTITY*O.UNIT_PRICE)/COUNT(DISTINCT  
O.ORDER_ID),2) AS AVG_REVENUE_PER_ORDER,  
CASE  
    WHEN SUM(O.QUANTITY*O.UNIT_PRICE)>=10000 THEN  
'EXCELLENT'  
    WHEN SUM(O.QUANTITY*O.UNIT_PRICE)>=6000 THEN 'GOOD'  
    WHEN SUM(O.QUANTITY*O.UNIT_PRICE)>=3000 THEN 'AVERAGE'  
    ELSE 'LOW'  
END  
AS CATEGORY_PERFORMANCE  
FROM CATEGORY_MASTER AS CG  
JOIN PRODUCT_MASTER AS P  
ON P.CATEGORY_ID=CG.CATEGORY_ID  
JOIN ORDER_DETAILS AS O  
ON O.PRODUCT_ID=P.PRODUCT_ID  
GROUP BY CG.CATEGORY_ID;  
  
SELECT * FROM MV_CATEGORY_SALE;  
  
EXPLAIN ANALYZE  
SELECT * FROM MV_CATEGORY_SALE;  
  
REFRESH MATERIALIZED VIEW MV_CATEGORY_SALE;
```